What Small Data Taught Me About Big Models: CNN Observations on the Intel Image Classification Dataset

Dundigalla Rithwik Mohan
Department of Information Technology
Chaitanya Bharathi Institute of Technology
Hyderabad, India
rizwik475@gmail.com

***Abstract*—This paper documents three unexpected observations made while applying classical CNN architectures—LeNet, AlexNet, ZF-Net, VGGNet, GoogLeNet, and ResNet—to the Intel Image Classification dataset using a balanced subset of 1,800 images across six scene categories. The first observation concerns the surprising severity of overfitting in AlexNet and how investigating the dataset size led to discovering that data augmentation improves the generalization gap without physically adding images. The second observation challenges a common assumption: pooling layers are *not* mandatory in CNN design, as demonstrated by ResNet's use of strided convolutions as an alternative downsampling mechanism. The third observation is that ResNet, despite being the simplest residual architecture tested, achieved the highest test accuracy (62.2%) while maintaining a lower training accuracy (55%)—a counterintuitive result that highlights the power of skip connections. These observations, grounded in hands-on experimentation, offer practical insights for practitioners working with limited-data image classification tasks.**

***Index Terms*—CNN, Data Augmentation, Overfitting, Pooling, Strided Convolution, ResNet, Skip Connections, Intel Image Classification, Small Dataset, Transfer Learning**

I. Introduction

Running well-known deep learning architectures on a new dataset rarely goes according to plan. Textbook descriptions of models like AlexNet or ResNet tend to focus on their performance on large-scale benchmarks such as ImageNet [2], leaving practitioners to discover on their own how these architectures behave under constrained conditions.

This paper reports three concrete observations that emerged from applying six classical CNN architectures to the Intel Image Classification dataset [1]—a six-category natural scene benchmark. Rather than presenting a comprehensive benchmark sweep, the focus here is on three moments of surprise during experimentation: an unexpected overfitting collapse in AlexNet that led to a deeper understanding of data augmentation; the realization that pooling layers are not a universal requirement in CNNs; and ResNet's counterintuitive pattern of higher test accuracy than training accuracy on a small dataset.

Each observation is described in the context of how it was discovered, what the underlying mechanism is, and what practical lesson it provides.

II. Dataset and Experimental Setup

The Intel Image Classification dataset [1] consists of RGB images originally sized 150×150 pixels across six scene categories: buildings, forest, glacier, mountain, sea, and street. For all experiments, a stratified balanced subset of 300 images per class (1,800 total) was used to maintain equal class representation. Images were resized to 32×32 pixels and normalized to $[0, 1]$. An 80/20 train-test split was applied throughout. All pre-trained architectures were implemented in TensorFlow/Keras and trained using the Adam optimizer and sparse categorical cross-entropy loss. Most architectures were trained for 8 epochs, while AlexNet was trained for 10 epochs without augmentation and 15 epochs with augmentation to better analyze overfitting behavior.

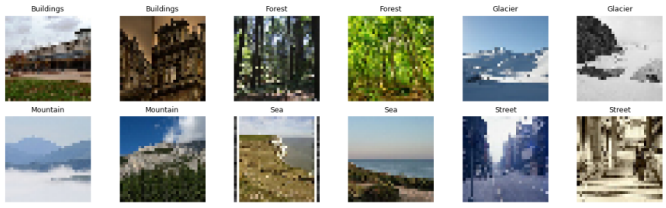

Fig. 1. Sample images from the Intel Image Classification dataset. A balanced subset of 1,800 images (300 per class) was used for all experiments.

III. Observation 1: Overfitting Reveals a Hidden Truth About Data Augmentation and Dataset Size

A. How This Was Discovered

When AlexNet was first trained on the 1,800-image subset without any augmentation, the results were alarming: training accuracy reached approximately 75.8% while test accuracy collapsed to just 46.9%—a gap of 28.9 percentage points (see Table I). In contrast, LeNet trained on the same data achieved 57% training accuracy and 61.7% test accuracy with no overfitting at all.

The obvious question was: *why is AlexNet, a more powerful model, performing worse on unseen data?* Investigating this

led to a closer look at the dataset size. With only 1,800 images across six classes (300 per class), the training set contains just 240 images per class after the 80/20 split. AlexNet—originally designed for 1.2 million ImageNet images [2]—has far more parameters than this data can reliably constrain.

TABLE I
ALEXNET PERFORMANCE BEFORE AND AFTER DATA AUGMENTATION

Condition	Train Accuracy	Test Accuracy
Without augmentation	75.8%	46.9%
With augmentation	61.7%	61.1%
Gap reduction	From 28.9% $\rightarrow \approx 0.6\%$	

B. What Data Augmentation Actually Does

After applying data augmentation (rotation $\pm 20°$, zoom $\pm 20\%$, width/height shifts, horizontal flips) and retraining AlexNet for 15 epochs, the results improved substantially: training accuracy was approximately 61.7% and test accuracy reached 61.1%. The gap was effectively eliminated, demonstrating near-perfect generalization with almost no train-test gap after augmentation.

The key realization here was that *data augmentation does not change the physical size of the dataset*. The folder on disk still contains the same 1,800 images. What augmentation does is expose the model to *virtually different images* at each training step by applying random geometric and photometric transformations on the fly. From the model's perspective, it is never shown the exact same image twice, which reduces memorization and encourages the learning of more general, transformation-invariant features.

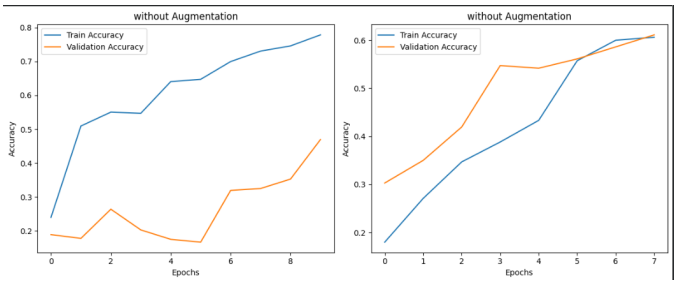

Fig. 2. AlexNet training vs. validation accuracy curves before (left) and after (right) data augmentation. Without augmentation, training accuracy rises to 75.8% while validation accuracy remains much lower at 46.9%, indicating severe overfitting. With augmentation, training and validation performance converge closely, with the gap reducing from 28.9% to approximately 0.6%, effectively eliminating overfitting.

C. Practical Lesson

When a large architecture overfits on a small dataset, the solution is not necessarily to collect more data—which is often impractical. Data augmentation acts as implicit regularization by synthetically expanding the diversity of training examples without adding a single new image to disk. This is especially important for pre-trained architectures with high parameter counts when applied to limited datasets.

IV. OBSERVATION 2: POOLING IS NOT MANDATORY — STRIDED CONVOLUTIONS ARE A VIABLE ALTERNATIVE

A. The Assumption

Going into this project, pooling layers (MaxPooling or AveragePooling) seemed like a fundamental, non-negotiable building block of any CNN. Every architecture covered in standard coursework—LeNet, AlexNet, VGGNet—uses pooling to reduce spatial dimensions between convolutional layers. It was reasonable to assume this was simply how CNNs work.

B. What ResNet Revealed

When implementing ResNet [3] for this study, the architecture uses *strided convolutions* rather than pooling layers for spatial downsampling in certain residual blocks. Specifically, a stride of 2 in a convolutional layer reduces the feature map dimensions by half—the same spatial effect as a 2×2 MaxPool—but unlike pooling, the strided convolution *learns* how to downsample rather than applying a fixed operation.

Table II summarizes the key differences between the two approaches.

TABLE II
MAXPOOLING VS. STRIDED CONVOLUTION FOR SPATIAL DOWNSAMPLING

Property	MaxPooling	Strided Conv.
Spatial reduction	Fixed (max)	Learned
Learnable parameters	None	Yes
Information loss	Higher	Lower
Used in	LeNet, AlexNet, VGGNet	ResNet
Preserves gradients	Partially	Better

C. Why This Matters

The distinction is not just architectural trivia. MaxPooling discards all but the maximum activation in each pooling window—a fixed, non-learnable operation that can cause information loss. Strided convolutions, by contrast, allow the network to *decide* which spatial information to preserve during downsampling, guided by the training loss. This is one reason ResNet's residual blocks can maintain cleaner gradient flow compared to pooling-heavy architectures.

D. Practical Lesson

Pooling is a design choice, not a requirement. Modern architectures increasingly favor strided convolutions because they give the network more control over feature extraction. For practitioners building custom CNNs, strided convolutions are worth considering—especially in architectures where gradient flow and information preservation are priorities.

V. OBSERVATION 3: RESNET'S TEST ACCURACY EXCEEDED ITS TRAINING ACCURACY — A COUNTERINTUITIVE RESULT EXPLAINED

A. The Surprising Numbers

After training all six architectures on the same 1,800-image dataset, ResNet produced the following results:

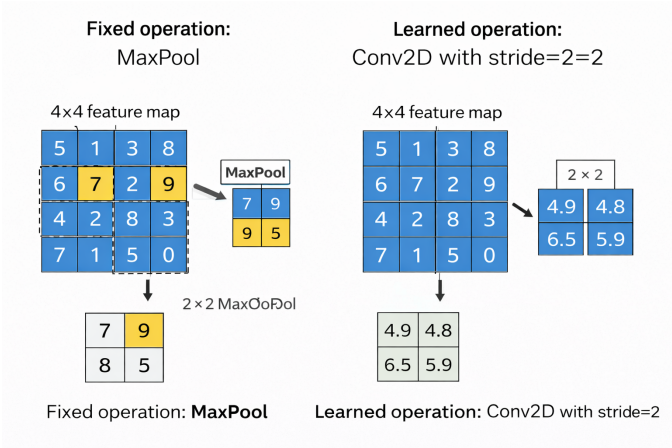

Fig. 3. Spatial downsampling via MaxPooling (fixed) versus strided convolution (learned). In MaxPooling, the maximum value from each region is selected, while strided convolution learns how to downsample using trainable weights. ResNet replaces pooling layers with strided convolutions for better feature learning.

- Training accuracy: $\approx 55\%$
- Test accuracy: $\approx$ **62.2%**

In most machine learning scenarios, training accuracy is *higher* than test accuracy because the model has been directly optimized on the training data. A model that performs *better on unseen data* than on its own training set is unusual and warrants explanation.

B. Why This Happened

Two mechanisms work together to produce this pattern:

1. Dropout during training. ResNet's architecture includes a Dropout(0.5) layer before the final dense layer. During training, 50% of neurons are randomly deactivated at each step, which artificially lowers training accuracy. During evaluation, all neurons are active, which restores the model's full representational capacity. This alone can create a training-test accuracy gap where test $>$ train.

2. Skip connections and generalization. ResNet's residual (skip) connections allow the gradient to flow directly from later layers back to earlier ones during backpropagation [3]. This solves the vanishing gradient problem that affects deeper networks without skip connections. On a small dataset, this means ResNet trains stably and avoids the erratic weight updates that plague deeper non-residual networks, resulting in a model that generalizes rather than memorizes.

C. Architecture Comparison

Table III shows the full comparison across all six architectures. ResNet achieves the highest test accuracy despite having a lower training accuracy than AlexNet and VGGNet.

D. Practical Lesson

A lower training accuracy does not mean a model is performing poorly. When Dropout is present and skip connections stabilize learning, the model can generalize far better than its training-time numbers suggest. On small datasets in particular,

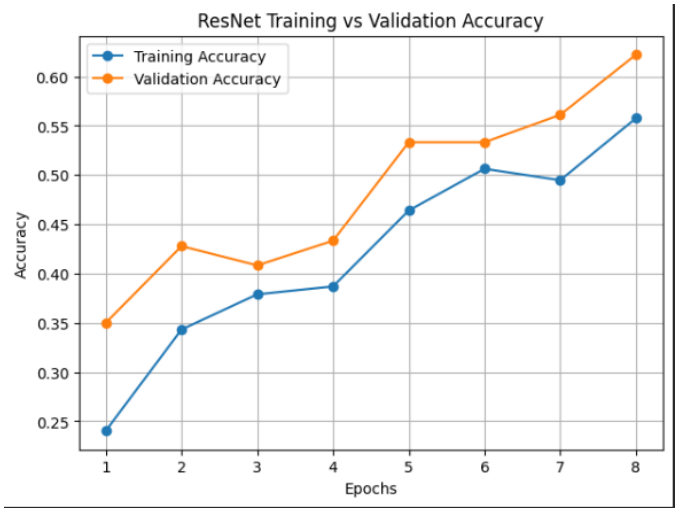

Fig. 4. ResNet training vs. validation accuracy over 8 epochs. The validation accuracy (62.2%) consistently exceeds training accuracy (55%), demonstrating strong generalization due to Dropout regularization and efficient gradient flow enabled by skip connections.

TABLE III
PRE-TRAINED ARCHITECTURE COMPARISON ON INTEL IMAGE CLASSIFICATION (1,800 IMAGES)

Architecture	Train Acc.	Test Acc.	Gap
LeNet-5	57.0%	61.7%	−4.7%
AlexNet*	61.7%	61.1%	+0.6%
ZF-Net	56.3%	56.9%	−0.6%
VGGNet	60.1%	59.1%	+1.0%
GoogLeNet	56.7%	56.3%	+0.4%
ResNet	55.0%	**62.2%**	−7.2%

*AlexNet result after data augmentation and longer training (15 epochs). Negative gap indicates test accuracy exceeded training accuracy.

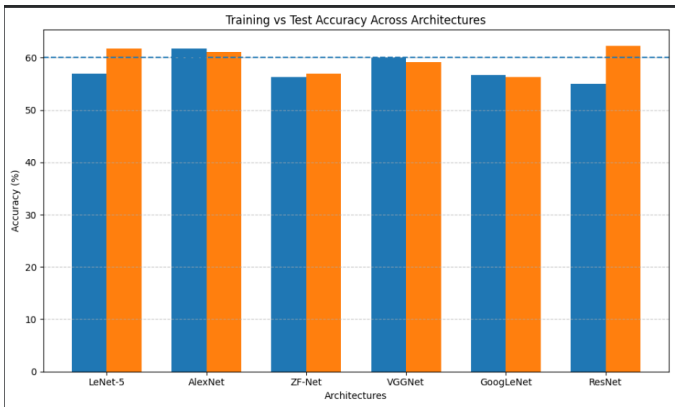

Fig. 5. Training vs. test accuracy comparison across all architectures. Each model shows paired bars for training and test accuracy. ResNet is the only architecture where test accuracy exceeds training accuracy, and it achieves the highest test performance overall. A reference line at 60% highlights the relative performance across models.

a modest training accuracy paired with high test accuracy is a sign of a well-regularized, robust model—not a failure.

VI. Discussion

The three observations documented here each challenge a common assumption held before beginning this project:

1) *"A larger model will perform better."* — AlexNet performed worse than LeNet without augmentation, and only improved after data augmentation demonstrated near-perfect generalization with almost no train-test gap.
2) *"CNNs need pooling layers."* — ResNet's use of strided convolutions shows that spatial downsampling is a design choice, and the learnable approach can outperform fixed pooling in terms of both information retention and generalization.
3) *"High training accuracy means good performance."* — ResNet's lower training accuracy paired with the highest test accuracy demonstrates that generalization metrics, not training metrics, are what matter for deployment.

Together, these observations reinforce a broader theme: *understanding why a result occurs matters more than the result itself*. The instinct to investigate the dataset size when AlexNet overfitted, rather than simply accepting the number, led to one of the most practically useful findings of the study.

VII. Conclusion

This paper presented three observations discovered during hands-on experimentation with CNN architectures on the Intel Image Classification dataset. First, AlexNet's severe overfitting on 1,800 images revealed that data augmentation does not physically expand a dataset—it creates virtual diversity that reduces memorization and achieves near-perfect generalization with almost no train-test gap. Second, implementing ResNet demonstrated that pooling layers are not mandatory: strided convolutions provide a learnable, information-preserving alternative for spatial downsampling that modern architectures increasingly prefer. Third, ResNet achieved the highest test accuracy (62.2%) while maintaining a lower training accuracy (55%), a counterintuitive but explainable result driven by Dropout regularization and skip-connection-aided gradient flow.

These findings underscore the value of curiosity-driven investigation in deep learning: the most useful insights emerged not from following the expected path, but from questioning results that did not match prior assumptions.

References

[1] Puneet Bansal, "Intel Image Classification Dataset," *Kaggle*, 2019. [Online]. Available: https://www.kaggle.com/datasets/puneet6060/intel-image-classification

[2] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," *Advances in Neural Information Processing Systems (NeurIPS)*, 2012.

[3] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.

[4] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," *Journal of Machine Learning Research*, vol. 15, pp. 1929–1958, 2014.

[5] J. T. Springenberg, A. Dosovitskiy, T. Brox, and R. Riedmiller, "Striving for simplicity: The all convolutional net," *ICLR Workshop*, 2015.

[6] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," *International Conference on Learning Representations (ICLR)*, 2015.